

Contracts Use Case Categorization

Document Number: P2185R0

Date: 2020-06-11

Authour: Caleb Sunstrum

Disclaimer

While the authour bears a professional relationship outside the Committee with the Chair of SG21, the categorization herein is the effort of the authour only and does not (necessarily) represent the views of the Chair.

Goals

With ~200 use cases collected by the authours of P1955R0 (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html>) (much thanks to them for this initial work), it's difficult to reason about let alone attempt to rank the relative importance of the various use cases!

This work is an attempt to categorize the various use cases in a way such that at each level of categorization the set of things to reason about is small enough to keep in one's working memory.

Notes

Contracts use cases fall into two primary categories:

- “What I want to use contracts *for*”
- “*How* I want to use contracts” (meta-Contracts)

The vast majority of provided use cases fall into the second category, and most of the former are non-controversial. It seems the majority of disagreement around Contracts relates to how the facility is specified and used as opposed to what we want to use it for!

There are also a few agendas at play, such as C-compatibility and elimination of macros.

Contracts Use Cases

“What I want to use contracts for”

```
namespace uses {  
    class Correctness;  
    class Optimization;  
    class Code-Analysis;  
}
```

Correctness

“I want to use Contracts to help ensure my program is correct”

```
class uses::Correctness {  
    class Class-Consistency {  
        class Derivation;  
        class Interface;  
        class Internal;  
    };  
    class Defensive-Coding {  
        class Safety;  
        class TDD;  
    };  
    class Validation;  
};
```

Correctness::Class-Consistency::Derivation

How base-class contracts interact with derived classes

`cppapi.class.preconditions` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.preconditions)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.preconditions](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.preconditions)): Ensure overriding methods have same or wider preconditions (see: Liskov substitution principle)

`cppapi.class.postconditions` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.postconditions)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.postconditions](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.postconditions)): Ensure overriding functions meet their base class postconditions when their base class preconditions are met (see: Liskov substitution principle)

`cppapi.class.variability` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.variability)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.variability](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.class.variability)): Allow overriding functions to have narrower preconditions/wider postconditions if I want to

`api.class.baseinterface` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.baseinterface)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.baseinterface](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.baseinterface)): Express a restriction on the protected interface of a type that derived types can depend upon: can mention only protected and public members, and is checked on entry and exit from this type's code

`api.class.baseinvariants` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.baseinvariants)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.baseinvariants](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.baseinvariants)): Check invariants on entry and exit of every protected method (when called from the derived type, not when one base member function calls another)

`api.class.basecalls` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.basecalls>):

Check invariants before and after every call to a virtual function (when calling to the derived type)

Correctness::Class-Consistency::Interface

Class contracts over the public interface of a class

`cppapi.invariants` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.invariants>):

Declare class invariants that all of my public functions need to maintain

`api.class.publicinterface` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publicinterface)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publicinterface](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publicinterface)): Express a restriction on the public interface of a type that all callers of the type can depend upon: can mention only public members, and is checked on entry and exit from this type's code

`api.class.publicinvariants` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publicinvariants)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publicinvariants](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publicinvariants)): Check invariants before and after every public method (when called from outside the type, not when one member function calls another)

`api.class.privateinvariants` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privateinvariants)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privateinvariants](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privateinvariants)): Check invariants on entry and exit of every public method (when called from outside the type, not when one member function calls another)

Correctness::Class-Consistency::Internal

Class contracts for internal consistency

`api.class.publiccalls` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.publiccalls>):

Check invariants before and after calling functions that are not part of this type (including virtual calls)

`api.class.privateinterface` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privateinterface)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privateinterface](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privateinterface)): Express an internal restriction on the private implementation of a type, can mention any member, and is checked on entry and exit from this type's code

`api.class.privatecalls` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.privatecalls>): Check invariants before and after calling functions that are not part of this type (including virtual calls)

Correctness::Defensive-Coding::Safety

Ensuring error paths have well-defined behaviour

`pdev.safety.isolation` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#pdev.safety.isolation>): Isolate safety checks from performance annotations

`pdev.safety.critical` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#pdev.safety.critical>): Retain checking even when optimizing with performance annotations

`crit.noassume` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.noassume>): Ensure checks will never be `__assume 'd/ __builtin_assume 'd` by the compiler as if they were facts injected into the program (otherwise, if such an assumption ever failed, I would be running a different program that is not equivalent to the one I wrote; assumptions can expand the set of possible executions by injecting facts not otherwise knowable to the compiler) (See also: Optimization::UB)

`crit.redundancy` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.redundancy>): Be able to express error handling that may be redundant with contract checking

`crit.interaction` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.interaction>): Not have contract build or run modes possibly be able to change or disable related error handling in any way

Correctness::Defensive-Coding::TDD

Using Contracts for Test-Driven Development

`adev.evolve` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#adev.evolve>): Assert against conditions I am aware of but not finished handling fully

Correctness::Validation

Using Contracts to validate program correctness

api.establish.check (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.check>):

Have validation inform me which output values are unexpected or invalid

api.establish.validate_invariants ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.validate_invariants)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.validate_invariants](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.validate_invariants)): Have validation inform me which class invariants are violated

api.establish.values (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.values>):

Have validation inform user which input values are unexpected or invalid

jdev.understand.violations ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.violations)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.violations](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.violations)): Be informed when my usage is out of contract

int.conform.postconditions ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conform.postconditions)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conform.postconditions](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conform.postconditions)): Verify results from a call are expected output values

Optimization

“I want to use Contracts to help optimize my program”

```
class uses::Optimization {  
    class Leverage;  
    class Overhead;  
    class UB;  
};
```

Optimization::Leverage

Using Contracts to improve optimization

dev.reason.confidence ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.confidence)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.confidence](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.confidence)): Express a spectrum of confidence in my annotations, from “unsure” and asking for validation, to “sure” and asking for some effect to be applied (eg. “maybe”, “definitely”, “assume” ‘something’) (See also: [Functionality::Expressivity::Meta-Information](#))

int.build.optimize (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.optimize>): Turn on run time optimization to leverage annotation assumptions

hardware.performance (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#hardware.performance>): Be able to design new hardware + optimizations, carefully dovetailed into one another, that depend on statically-unprovable facts being annotated in the code

pdev.speed (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#pdev.speed>): Annotate my code with assumptions, likelihoods, or reachability information that a tool might not be able to deduce, but that I would be confident of

pdev.morespeed (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#pdev.morespeed>): Be able to give statically-unprovable facts to current and novel optimizers in terms of semantics my program does not depend-on but optimizers can't figure out

Optimization::Overhead

Run-time overhead of Contracts

large.perfcontrol.runtime (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.perfcontrol.runtime>): Constrain the set of runtime checks according to their performance overhead

embedded.nochecking (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#embedded.nochecking>): Remove all checking and diagnostic (eg. source location) overhead entirely from the final binary

int.runtime.unchecked (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.runtime.unchecked>): Turn off run time checking to remove checking overhead

Optimization::UB

Undefined behaviour introduced by Contracts

pdev.footgun (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#pdev.footgun>): Accept responsibility for a malformed program that might result from eventually false information given by my annotations

crit.noassume (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.noassume>): Ensure checks will never be `__assume 'd/` `__builtin_assume 'd` by the compiler as if they were facts injected into the program (otherwise, if such an assumption ever failed, I would be running a different program that is not equivalent to the one I wrote; assumptions can expand the set of possible executions by injecting facts not otherwise knowable to the compiler) (See also: [Correctness::Defensive-Coding::Safety](#))

Code Analysis

“I want to use Contracts to improve the results of tooling”

```
class uses::Code-Analysis {  
  class Tooling-Annotations {  
    class Accuracy;  
    class Ignorability;  
  };  
  class Code-Generation;  
  class Verification;  
};
```

Code-Analysis::Tooling-Annotations::Accuracy

Using Contracts to improve the accuracy of tools

`dev.tooling` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.tooling>): Expose annotations to tools that might leverage them (eg. code linter, static analyzer, semantic prover, compiler sanitizer, binary analyzer, code reviewer, etc.)

`analysis.information` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.information>): Be able to hint to the analyzer information it may be unable to deduce from source code alone (eg. `5 / opaque(); [[ opaque() != 0]]`)

Code-Analysis::Tooling-Annotations::Ignorability

Make it feasible for tooling to ignore misunderstood annotations

`qdev.tooling` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.tooling>): Signify subset of individual annotations to be consumed by a specific kind of verification tool

`qdev.tooling.control` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.tooling.control>): Signify subset of individual annotations to be consumed by a specific instance of verification tool

`qdev.tooling.undefined` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.tooling.undefined>): Use predicates that may not be understood by all instances of verification

`qdev.tooling.undefinedkinds` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.tooling.undefinedkinds>): Use predicates that may not be understood by all kinds of verification

Code-Analysis::Code-Generation

Integrating the results of tooling into the program

`analysis.runtime` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.runtime>): Have runtime checks generated by the tool

`analysis.optimization` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.optimization>): Have runtime optimizations generated by the tool

`qdev.tooling.behavior` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.tooling.behavior>): Integrate the results of that static checker into how my program behaves in different ways: assume proven predicates, make unprovable predicates ill- formed, etc.

Code-Analysis::Verification

Using tooling to verify program correctness

`int.consistency` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.consistency>): Verify all annotations are globally consistent when integrated

`qdev.correctness` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.correctness>): Signify the predicates that should be verified by an analysis tool

`analysis.symbolic` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.symbolic>): Have symbolic proofs for soundness and consistency performed before compile time

`analysis.compiletime` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.compiletime>): Have code source, AST, or inclassion inspection during compile time

`analysis.binaries` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.binaries>): Have binary inspection after compile time

Contracts Meta Use Cases

“How I want to use contracts”

```
namespace meta {  
    class Controllability;  
    class Functionality;  
    class Teachability;  
    class Agenda-Advancement;  
};
```


Controllability

“How I want to control Contracts”

```
class meta::Controllability {
  class Control {
    class Maintenance;
    class Security;
    class Run-time;
    class Selection;
    class Source;
  };
  class Granularity {
    class Build;
    class Debug;
    class Source;
  };
  class Reporting {
    class Customization;
    class Stripping;
  };
  class Testing {
    class Override-Checking;
    class Override-Handling;
  };
  class Violation-Handling {
    class Continuation;
    class Customization;
  };
};
```

Controllability::Control::Maintenance

How Contracts should affect code maintenance

int.build.binarycounts (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.binarycounts>):

Only be required to manage a small, common set of build/link configurations

lib.maintenance.noconfig ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.maintenance.noconfig)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.maintenance.noconfig](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.maintenance.noconfig)): Not require extra build steps to be documented

lib.maintenance.nowhining ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.maintenance.nowhining)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.maintenance.nowhining](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.maintenance.nowhining)): Not have users complain about my product due to modifications of annotations resulting from their build configuration

`lib.integration.noconfig` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.integration.noconfig>): Not require extra build steps to be learned or performed

Controllability::Control::Security

Using Contracts in a secure way

`sec.noattacks` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#sec.noattacks>): Be unable to insert code paths (eg. violation handlers) at run time (eg. build time only)

`sec.certify` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#sec.certify>): Have build tool only link to a preapproved violation handler

Controllability::Control::Run-time

Run-time control over contracts

`int.control.runtime` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.control.runtime>): Turn checks on at run time

`int.control.subsets.runtime` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.control.subsets.runtime>): Turn on any subset of individual (call site) checks on at run time

`int.build.common` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.common>): Be able to use the same executable regardless of contract enforcement mode

Controllability::Control::Selection

Build-time control over contracts

`cppdev.build.legacy` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.build.legacy>): Have annotations affect executions depending on my existing build modes (eg. Debug or Release modes in VS)

`int.control.build` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.control.build>): Turn checks on at build time

`int.build.unchecked` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.unchecked>): Turn off build time checking to remove checking overhead

`crit.control` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.control>): Be able to control the configuration of contracts from a central point

Controllability::Control::Source

Source-level control over contracts

`lib.integration.nowhining` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.integration.nowhining)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.integration.nowhining](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#lib.integration.nowhining)): Not have my users accidentally modify my careful annotations

`sdev.control` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#sdev.control>): Disable remapping of semantics on stable and correct individual contracts

`crit.production.checking` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.production.checking)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.production.checking](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.production.checking)): Be able to continue to run checks in a production environment (even after formal testing is complete)

`crit.more.coverage` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.more.coverage>): Be able to run checks in a production environment that are considered “cheap” compared to the expected cost of entering an invalid state

Controllability::Granularity::Build

Granular control of contracts at build-time

`int.conrol.subsets.build` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conrol.subsets.build)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conrol.subsets.build](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conrol.subsets.build)): Turn on any subset of individual (call site) checks on at build time

`int.control.subsets` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.control.subsets>): Have a way to audit (named or semantic) subsets of checks for various deployments

`large.perfcontrol.build` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.perfcontrol.build)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.perfcontrol.build](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.perfcontrol.build)): Constrain the set of built time checks according to their performance overhead

`int.build.minimize` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.minimize>):

Disable library postconditions, asserts, and invariants, without disabling library preconditions (assuming the library is tested and stable and my code is not)

`qdev.checkall` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.checkall>): Ensure all checks (pre, post, assert, invariant) are enabled

Controllability::Granularity::Debug

Granular control over contracts for debugging purposes

large.complex (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.complex>): Have composable and fine grained control over which checks are run, without requiring source code changes. Specifically the checks for only one function or some grouping of functions

int.build.control (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.control>): Enable checks only within a selected library

int.build.control2 (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.control2>): Enable checks on multiple libraries simultaneously

int.debug.callsites (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.debug.callsites>): Enable checks only on selected call sites

large.simulation.disable (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.simulation.disable>): Optionally disable checking on a subset of individual annotations

large.simulation.enable (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.simulation.enable>): Optionally allow checking of a subset of individual annotations to fail and access its recovery path

large.simulation.ignore (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.simulation.ignore>): Optionally allow checking of a subset of individual annotations to fail and continue failing

Controllability::Granularity::Source

Granular control over contracts at the source level

large.critical (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.critical>): Control whether checks are run based on where they are being called from

large.separability (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.separability>): Be able to include distinct clauses for each parameter or invariant with their own individual failure or build controls

Controllability::Reporting::Customization

Control over how contract violations are reported

int.violations.transmit (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.violations.transmit>): Transmit check failure information in environment-specific ways (logs, email, special hardware traps, popup windows, blazing sirens, etc).

`qdev.fuzz.testing` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.fuzz.testing>): Log all predicate failure during fuzz testing

`large.scalability` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.scalability>): Be able to log violations in my organization specific format

Controllability::Reporting::Stripping

Control over what information is retained/reported

`embedded.nologging` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#embedded.nologging>): Remove all logging and diagnostic (but not checking) overhead from the final binary

`embedded.minimize` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#embedded.minimize>): Remove all but the most important diagnostic overhead from the final binary

`bdev.confidentiality` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#bdev.confidentiality>): Not expose diagnostic information (source location, expressions, etc.) in the software I deliver to clients, even when I choose to have contracts enforced in the software I deliver

Controllability::Testing::Override-Checking

Override how/when contracts are checked for testing purposes

`api.class.testing` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.class.testing>): For every member or friend function in my class, run my unit test framework with checking enabled for every assertion at the point where it is written, and check every postcondition at every non-exceptional exit, and test my class invariants on entry and exit from this type's code

`int.testing.control` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.testing.control>): Selectively enable checking for a set of functions which could name either an individual function or an overload set

`int.testing.controltypes` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.testing.controltypes>): Selectively enable checking for a set of types and all their members

`int.testing.transitivity` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.testing.transitivity>): Selectively enable checking for a set of types and all their transitively nested types and members

`int.testing.modules` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.testing.modules>): Selectively enable checking for a translation unit or module and all (non transitive) types and functions within

`crit.testing` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.testing>): Be able to run both success and failure branches in my test environment

`large.narrowing` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.narrowing>): Be able to narrow individual contract so it fails in testing not in production

Controllability::Testing::Override-Handling

Override how contract violations are handled for testing purposes

`qdev.testing` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.testing>): Override failure handler to trigger test failure instead of termination

`qdev.handler.testing` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#qdev.handler.testing>): Have a way to run handler on all combinations of available build modes

Controllability::Violation-Handling::Continuation

Control over whether contract violations can be recovered from

`sdev.quality` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#sdev.quality>): Discourage reliance on observable out-of-contract behavior by causing check failure to hard stop program or build

`sdev.maturity` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#sdev.maturity>): Disable continuation on violation of stable and correct individual contracts

`large.observation` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.observation>): Have failed individual checks from existing code optionally warn instead of hard stop

`large.introduction` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.introduction>): Have failed checks from a new library optionally warn instead of hard stop

`large.newenvironment` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.newenvironment>): Have failed checks caused by a change in environment optionally warn instead of hard stop

`large.newcompiler` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.newcompiler>): Have failed checks caused by a change in compiler optionally warn instead of hard stop

`large.nogoingback` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.nogoingback>): Have trusted contracts fail fast and hard stop

`crit.recovery` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.recovery>): Have access to a recovery path after contract violation

Controllability::Violation-Handling::Customization

Being able to customize how contract violations are handled

`dev.reason.behaviorcontrol` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.behaviorcontrol>): Have the effect of annotations on executions be user controllable (ie. decide whether “cheap” checks or “critical” terminates)

`int.violations.custom` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.violations.custom>): Install custom violation handler where I can inject custom logic to trap errors

`int.violations.common` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.violations.common>): Be able to override how library violations are handled in the combined software to point into my handling code

`int.violations.override` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.violations.override>): Be able to define and override violation handler via source code

`crit.locality` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.locality>): Couple recovery path to a specific contract within the source

Functionality

“How I want to write Contracts”

```

class meta::Functionality {
  class Expressivity {
    class Annotations;
    class Form;
    class Meta-Information;
  };
  class Flexibility {
    class Code;
    class Evolution;
    class Location;
    class Unevaluated;
    class Usage;
  };
  class Interactions {
    class Code-Generation;
    class Extensibility;
    class Intercompatibility;
    class Predictability;
  };
  class Reporting {
    class Actionable;
    class Visible;
  };
};

```

Functionality::Expressivity::Annotations

Describe and enforce traits

`api.extend.exceptionsafety` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.exceptionsafety>): Annotate operations as being exception safe

`api.extend.threadsafety` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.threadsafety>): Annotate operations as being thread safe

`api.extend.atomicity` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.atomicity>): Annotate operations as being atomic (ie. all or no changes become visible)

`api.extend.realtime` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.realtime>): Annotate operations as real-time (ie. guaranteed to complete within a time frame)

`api.extend.determinism` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.determinism)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.determinism](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.determinism)): Annotate operations as being deterministic (ie. same outputs for same inputs)

`api.extend.purity` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.purity>):

Annotate operations as functionally pure (ie. no side effects)

`api.extend.sideeffects` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.sideeffects)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.sideeffects](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.sideeffects)): Annotate operations as having global side effects (ie. write to singleton, file, network, or database)

`api.extend.complexity` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.complexity)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.complexity](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.extend.complexity)): Annotate algorithmic complexity

Functionality::Expressivity::Form

How Contracts are specified

`api.establish.preconditions` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.preconditions)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.preconditions](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.preconditions)): Have contracts specify their pre-conditions as logical predicates

`api.establish.invariants` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.invariants)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.invariants](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.invariants)): Have contracts specify their class invariants as logical predicates

`api.establish.postconditions` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.postconditions)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.postconditions](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.postconditions)): Have contracts specify their post-conditions as logical predicates

`arch.complete` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#arch.complete>): Specify preconditions/postconditions/assertions/invariants that express my expectations about the expected valid state of my program in the form of compilable boolean expressions, that can be checked statically or dynamically (as opposed to disjointed state where these features are factored into bits)

Functionality::Expressivity::Meta-Information

Encoding information about the contract itself

`dev.reason.confidence` ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.confidence)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.confidence](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.confidence)): Express a spectrum of confidence in my annotations, from “unsure” and asking for validation, to “sure” and asking for some effect

to be applied (eg. “maybe”, “definitely”, “assume” ‘something’) (See also: Optimization::Leverage)

`dev.reason.importance` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.importance>): Express a spectrum of importance of my annotations, from “critical” (eg. bring the system down) to “minor” (eg. lead to a slower fallback)

`dev.reason.cost` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.cost>): Express a spectrum of expected cost at compile or runtime of my annotations, from “unrunnable” to “expensive” to “cheap”

Functionality::Flexibility::Code

How a contract can be written

`cppdev.syntax.reuse` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.syntax.reuse>): Have annotations use my custom types or functions

`api.express.values` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.express.values>): Make reference to either the values of my inputs, or other in-scope identifiers

`api.establish.changedvalues` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.changedvalues>): Make reference to the before and after values of in-out variables (ie. passed by pointer or reference) in post-conditions

`api.establish.changedmembers` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.changedmembers>): Make reference to the before and after values of mutable class members (eg. `new_size = old_size+1` after `push_back`) in post-conditions

`api.establish.changedstate` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.changedstate>): Make reference to the before and after values of global state (eg., `global >= old(global) + 1`) in post-conditions

`api.contract.private` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.contract.private>): Be able to access private implementation details of the class so I don’t have to widen public interface to declare predicates

`cppapi.variadic` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.variadic>): Allow predicate (fold) expansion

Functionality::Flexibility::Evolution

How a contract can be evolved

`dev.adapt` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.adapt>): Be able to easily change my confidence, importance, or other properties of my annotations over time

Functionality::Flexibility::Location

Where a contract can be written

`int.build.headeronly` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.headeronly>):
Use contract-enabled header-only libraries

`dev.reason.knowl` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.knowl>):
Annotate my program anywhere in the code with my current understanding of its structure or execution

`cppdev.location` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.location>): Use same source file for both code and annotations

`api.contract.interface` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.contract.interface>):
Declare contract when I declare the function

`api.contract.redeclaration` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.contract.redeclaration>): Place function contract conditions on any declaration (e.g., on redeclarations at the bottom of the header, or on the definition in an implementation file, where they are less distracting).

`cpplib.headeronly` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cpplib.headeronly>): Be able to ship header only library

`cpplib.insulation` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cpplib.insulation>): Insulate contract conditions with the function definition, or insulate only the definition while putting contract conditions on a redeclaration - visible to static analysis tools in all TUs.

`api.communicate.inputsoutputs` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.communicate.inputsoutputs>): Document the expected inputs and expected outputs on my interface

Functionality::Flexibility::Unevaluated

Expressing unevaluated/unevaluable contracts

`api.express.runnability` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.express.runnability>): Be able to use a predicate that is not evaluated at runtime, because it might be unsafe to run or have stateful side effects

`api.express.undefined` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.express.undefined>): Be able to use a predicate that doesn't have a definition, because it hasn't been written yet, or is infeasible to run

`api.express.uncheckable` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.express.uncheckable>): Be able to use a predicate that is not evaluated, because it is simply a semantic placeholder for a tool

`api.express.unimplementable` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.express.unimplementable>): Be able to use a predicate that cannot have a complete definition, because it is inexpressible in the language

Functionality::Flexibility::Usage

Extended uses of Contracts

`cppapi.contracts.async` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.contracts.async>): Express contracts on callbacks such as `std::function`, function pointers, or references to functions, lambdas, or function objects

`cppapi.contracts.exception` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppapi.contracts.exception>): Express contracts on exceptional exit

`wg21.otherfeatures` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#wg21.otherfeatures>): Be able to use contract-like syntax on past or present runtime checkable language features such as switches, pattern matching, etc. or what might happen on signed integer overflow, etc. This might allow configuration of trapping, logging, or assuming in other areas of language UB.

Functionality::Interactions::Code-Generation

How Contracts affect code generation

`int.build.binaries` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.build.binaries>): Use contract-enabled binary libraries

`cppdev.debugger` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.debugger>): Have runtime able to launch a debugger from an annotation if necessary

Functionality::Interactions::Extensibility

How much room the Standard leaves for implementation extensions to Contracts

`compiler.benice` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#compiler.benice>): Maximize implementation freedom by limiting what is strictly required by the standard

Functionality::Interactions::Intercompatibility

How Contracts interact with other languages/language components

`cppdev.modules` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.modules>): Be interoperable with modules

`cppdev.coroutines` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.coroutines>): Be interoperable with coroutines

`cppdev.concepts` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.concepts>): Be interoperable with concepts

`ccppdev.interop` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#ccppdev.interop>): Not lose contracts when crossing languages

`cdev.cppinterop` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cdev.cppinterop>): Expose my contracts to C++ developers through `extern "C"` declarations of my functions

`api.coroutines` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.coroutines>): Define and check pre and post conditions [on a coroutine] as I would a regular function

`api.coroutines.invariants` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.coroutines.invariants>): Define and check invariants over all entry and exit points from a coroutine (to its awaiter or promise)

Functionality::Interactions::Predictability

Ensuring that program behaviour is predictable

`dev.reason.behavior` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.behavior>): Have annotations affect the execution of my program in accordance with my expectations

`dev.reason.sideeffects` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.reason.sideeffects>): Ensure annotations do not substantially change the meaning of my program whether enabled or disabled

`crit.noundef` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#crit.noundef>): Have contract violation at run-time always have well-defined behavior

Functionality::Reporting::Actionable

How actionable a violation report is

`api.establish.responsibility` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.establish.responsibility>): Inform users which errors are the responsibility of the caller, and which are the callee

`api.resp.preassert` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.resp.preassert>): Annotate assertions inside function bodies that indirectly test preconditions (such as malformed data discovered while performing the algorithm) should be reported to the caller as precondition failures

`int.violations.information` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.violations.information>): Be informed what check failed, when, where, and how

`jdev.understand.buildfailures` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.buildfailures>): Know why my software is not building

`jdev.understand.aborting` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.aborting>): Know why my software is aborting

`jdev.understand.omniscience` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.omniscience>): Know why my software is out of contract

Functionality::Reporting::Visible

How visible a contract violation is

`int.conform.violation` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#int.conform.violation>): Be informed any time an interface's contract is violated

`jdev.understand.buildviolation` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.buildviolation>): Know that my program or build was halted due to contract violation

Teachability

“How I want to teach Contracts”

```
class meta::Teachability {  
  class Reasoning {  
    class Composeable;  
    class Readable;  
  };  
  class Simplicity {  
    class Build;  
    class Code;  
    class Conceptual;  
  };  
  class Consistency;  
  class Documentation;  
};
```

Teachability::Reasoning::Composeable

Have Contracts be a set of building blocks that can be composed in any way

`teach.layering` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#teach.layering>): Support the ability for advanced uses of contracts to be distributed across many different courses in a C++-focused computer science curriculum.

`jdev.understand.contracts` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.contracts>): A uniform, fluent description of expected input values, expected output values, side effects, and all logical pre and post conditions

Teachability::Reasoning::Readable

Have Contracts be readable

`dev.parsable` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.parsable>): A syntax that can both be parsed and can be reasoned about semantically

`dev.readable.priority` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.readable.priority>): Have my contract specification to be visually primary, and secondary information (syntax, hints, roles, levels, etc.) to not be distracting

dev.readable.keywords ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.readable.keywords)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.readable.keywords](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.readable.keywords)): Have annotation keywords or names with intuitive, clear, and unambiguous meanings

jdev.understand.keywords ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.keywords)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.keywords](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.keywords)): Have keywords with precise and unambiguous meanings

Teachability::Simplicity::Build

Building code with Contracts should be simple and easy

teach.lifecycle (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#teach.lifecycle>):

Demonstrate mock lifecycle by switching simple compiler flags to control which checks are enabled

teach.dumbstudents (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#teach.dumbstudents>):

Have examples that are easy to build without digression into build systems

Teachability::Simplicity::Code

Writing Contracts should be simple and easy

dev.readable.syntax (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#dev.readable.syntax>):

Have annotations with a succinct and elegant syntax

jdev.understand.all (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.understand.all>): Be

able to build a program with contracts after reasonably short tutorial

adev.fast (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#adev.fast>): Be able to write and

modify contracts quickly without heavy boiler plate or up front cost

Teachability::Simplicity::Conceptual

Concepts for Contracts should be simple and easy to grasp

teach.teachable (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#teach.teachable>): Have

simple explanation of assertions and their use to support simple programming tasks, including debugging erroneous programs.

teach.bestpractices (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#teach.bestpractices>): Be

able to express defensive programming, programming by contract, and test driven development to introductory students

compiler.best (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#compiler.best>): Have a clear and simple specification that meets clear need

wg21.everythingelse (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#wg21.everythingelse>): Have a clear way to understand how contracts will interact with the standard library

Teachability::Consistency

Have Contracts be consistent

cppdev.syntax.familiar (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.syntax.familiar>): Have annotations use familiar syntax

cppdev.syntax.cpp (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.syntax.cpp>): Have annotations use C++ syntax

test.standardized (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#test.standardized>): Not rely on custom libraries or proprietary extensions

teach.portable (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#teach.portable>): Have examples compilable by a standard compiler on any system

Teachability::Documentation

Have useful documentation

sdev.bestpractices (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#sdev.bestpractices>): Demonstrate best practice in defensive programming

jdev.bestpractices (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#jdev.bestpractices>): Learn about software best practices by example

Agenda Advancement

“How I want to advance a particular agenda”

```
class meta::Agenda-Advancement {
    class C-Compatibility;
    class CPP-Adoption;
    class Macro-Elimination;
    class Unification;
};
```

Agenda-Advancement::C-Compatibility

Specifying Contracts in a way that makes it compatible with C

`cdev.contracts` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cdev.contracts>): Specify contracts in a way standardizable as part of the C language

`cdev.identifiers` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cdev.identifiers>): Use contracts with macro-safe keywords that are reserved C names (i.e., `_Pre` , `_Post` , `_Assert` , etc.)

`cdev.violationhandler` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cdev.violationhandler>): Have a common violation handler for both violated C and C++ contracts

`cdev.ignorable` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cdev.ignorable>): Make all contract semantics optional (so as not to change WG14-N2385 6.7.11 p2)

Agenda-Advancement::CPP-Adoption

Specifying Contracts in a way that makes it easier to adopt newer C++ editions

`analysis.legacy` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#analysis.legacy>): Be able to map pre-existing contract features in tools to a standardized language syntax

`large.modernize` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.modernize>): Introduce standardized contracts to replace my macro-based contracts

`large.stillmacros` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#large.stillmacros>): Have my existing macro-based facilities interoperate smoothly with standardized contracts so I can do the migration gradually

Agenda-Advancement::Macro-Elimination

Specifying Contracts in a way that doesn't encourage usage of macros

`cppdev.syntax.macros` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.syntax.macros>): Minimize use of macros

`arch.nomacros` (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#arch.nomacros>): Express assertions in a way that does not rely on C macros (i.e., there is no valid technical reason for a programmer not to use the new way, including space, time, tooling, and usability/complexity reasons, compared to C's `assert` macro)

Agenda-Advancement::Unification

Specifying Contracts in a way that allows other existing parts of C++ to be re-written using Contracts

cppdev.existing.std (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#cppdev.existing.std>):

Codify existing exposition-only standard library requirements

api.contract.errorhandling ([http://www.open-](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.contract.errorhandling)

[std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.contract.errorhandling](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html#api.contract.errorhandling)): Replace uses of error handling to express contract violation (eg. `operator[](size_t n) noexcept [[pre: n < size()]]` instead of throwing)

Epilogue

Caveats

The use cases are not mine, and I don't claim to fully and accurately understand all the implications and nuance of each one. The categorization is a best effort and is likely to change based on feedback.

Thanks

Thanks to Joshua Berne, Timur Doumler, Andrzej Krzemieński, Ryan McDougall, and Herb Sutter for their work collecting all of these use cases and attempting to distill their essences in P1995R0 (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html>), which this work builds on top of.